

Microarchitectural Side-Channel Mitigations for Serverless Applications

Yayu Wang

The University of British Columbia

Aastha Mehta

The University of British Columbia

1 Introduction

Serverless applications consist of small functions that communicate with each other via network APIs. Serverless platform providers package each function of an application in a separate container and manage scheduling and scaling of the functions based on incident workload. For efficient resource utilization on the platforms, providers often colocate the functions (containers) of multiple tenants on the same physical server. However, this raises concerns of side-channel leaks between functions of mutually distrusting tenants.

In this work, we focus on mitigating a class of microarchitectural side-channel leaks—via timing and access patterns—in serverless applications. We consider a colocated (unprivileged) adversary, which can measure the use of CPU, memory, and caches by the *victim* application [9, 11, 15, 18, 19].

Techniques to mitigate such leaks include coarsening timing measurements [4] and partitioning shared resources, such as caches [10, 14] and cores [16]. However, coarsening timing can break the functionality of legitimate applications, while resource partitioning approaches may not scale to serverless platforms, which are optimized for high resource utilization through sharing of infrastructure among tenants.

A principled approach to mitigating such side-channel leaks, which has been explored in the context of cryptographic applications, ensures that the application satisfies a formal property of *constant-time*. Formally, a constant-time (CT) program satisfies three properties; it has no (i) secret-dependent branches, (ii) secret-dependent memory accesses, and (iii) secret operands in variable-time instructions.

Unfortunately, prior CT solutions are insufficient to mitigate side-channel leaks of sensitive information in serverless applications due to several reasons. First, it is impractical to restrict the programming features used in serverless applications, thus precluding application development in DSLs [1]. Secondly, several solutions rely on developers’ annotations in the source code to guide the enforcement of CT property [3, 6, 12, 17]. However, adding annotations is a cumbersome task [8]. Other tools verify if a program satisfies CT property [2, 7] but do not generate a program with CT property. Finally, all these tools are designed for programs written in C-style languages that are compiled to native code, while serverless applications are often written in dynamically typed languages (e.g., JavaScript, Python) and executed within language runtimes. Thus, even if the application is transformed to ensure CT property at the source code or the IR level, it does not ensure CT execution within the runtime.

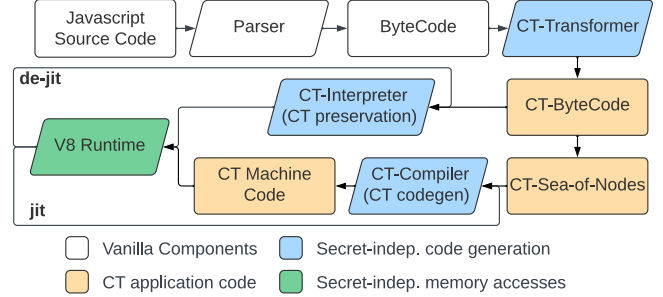


Figure 1. V8 workflow with constant-time transformations

Specifically, the interpreter or the JIT compilation toolchain may itself introduce side-channel vulnerabilities during the execution or the code generation phase [5].

2 Our solution

Our goal is to provide side-channel mitigations for serverless applications while minimizing developers’ burden for incorporating the solutions. Towards this end, we present **Scouti**, the first tool for JS applications, which provides mitigations within off-the-shelf V8 engine¹. At a high level, **Scouti** automatically transforms JS applications to enforce CT execution w.r.t application secrets.

Privacy goals. Our goal is to protect the secret inputs processed by the applications, e.g., login credentials, online transactions, and users’ personally identifiable information. However, we assume the source code and input metadata (e.g., size, type) are public, since an adversary can acquire this information through other means.

Challenges. Building CT transformations for serverless applications involves several challenges. **C1.** Existing tools that generate CT executables assume that most native instructions execute in a fixed, secret-independent time and require modifying the implementations of only a few variable-time operations (e.g., division, floating point and vector operations) [6]. Thus, generating CT code requires straightforward implementation within the IR layer in the tools. In a language runtime, non-CT behavior can arise in several places and could be exploited at runtime to leak application secrets. Specifically, the interpretation of several instructions exhibits non-CT behavior. For instance, an object attribute access in the source (`object[key]`) translates to a `GetKeyedProperty` opcode, which looks up the object’s attribute dictionary using (potentially secret) key. Furthermore, JIT compilation, compile-time and runtime optimizations may invalidate CT

¹We do not consider an approach based on WebAssembly [17], since transforming a program from a dynamically typed language to a statically typed language is cumbersome and defeats the convenience of development.

```

void* linearize_reference(void* o1, void* o2) {
    ref = (true_mask & (CAST<uint64_t>(o1))) |
          (false_mask & (CAST<uint64_t>(o2)));
}
void* linearize_object(void* o1, void* o2) {
    for (int i = 0; i < o1.size; ++i) {
        o[i] = (true_mask & CAST<uint8_t>(*o1[i])) |
              (false_mask & CAST<uint8_t>(*o2[i]));
    }
}

```

Figure 2. CT-Transformer: Reference, object linearization guarantees. All these components within a language runtime need to be adapted to ensure CT execution of the application.

C2. Existing solutions generate CT executables statically, while for applications written in JS-like languages generate executable code during deployment. Thus, ensuring CT transformations within language runtimes would incur runtime overheads, which need to be minimized.

C3. Unlike crypto, serverless applications use complex data structures (e.g., map and list), and looking up specific elements in these datastructures leads to pointer chasing. Hiding the memory access patterns of secret lookups on these datastructures would be challenging due to dynamic typing and dynamic memory allocation for such datastructures.

Key ideas. Mitigating side channels requires ensuring: (i) the application code is free of side channels, (ii) the V8 engine generates executable code without introducing new side channels. Fig 1 shows an overview of the steps involved in satisfying these requirements. CT-Transformer generates CT bytecode by linearizing the secret-dependent branch paths (similar to [3, 6]) and linearizing the indirect function calls based on secret lookups. Here, we make a key contribution by generalizing the branch transformation technique from prior work to enable linearization of code involving arbitrary data types beyond integers, such as strings and objects. Specifically, we enable linearization of accesses to references and objects (Fig 2). For indirect function calls, we use information-flow tracking to find all potential functions from a function table, when a function is invoked based on a secret lookup, and execute all the potential functions.

V8 preserves CT guarantees while transforming the CT-bytecode to the sea-of-nodes IR, since it does not access any secrets. For CT execution, we linearize the bytecode handlers in the interpreter (CT-Interpreter) to preserve the CT property when interpreting the bytecode. We also modify the JIT compiler (CT-compiler) to generate CT machine code from the sea-of-nodes IR. We disable or modify the compiler optimization passes that violate CT guarantees. Finally, we modify the V8 runtime to make the application's memory accesses secret-independent during code execution.

3 Current status and future work

Scooti demonstrates the feasibility of achieving automatic constant-time guarantees in language runtimes. Although it uses techniques from prior work, we demonstrate how

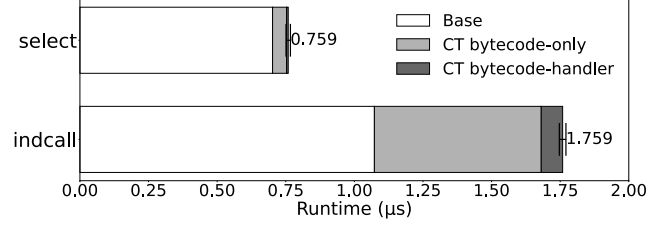


Figure 3. Running time of *select* and *indcall*

to use those techniques in a composable manner to achieve constant-time guarantees in JS runtime, which is generalizable to other runtimes as well. Currently, **Scooti** does not handle multi-threading, I/O, and secret-bounded loops.

We have a partial implementation of CT-Transformer. Of the 176 V8 bytecode instructions, we manually verified that 48 instructions covering basic register, arithmetic and comparison operations are already constant time. We transformed one instruction, `GetKeyedProperty`, and added one new constant-time instruction `CTSelect` to substitute 11 control flow related instructions.

We implement two microbenchmarks, *select* and *indcall* (Fig 4). We manually generate CT bytecode for the two functions. For each function, we execute the original bytecode with unmodified V8 (base) and the CT bytecode with the unmodified V8 (CT bytecode-only) as well as with modified interpreter handlers, if any involved (CT bytecode-handler). Fig 3 shows the average runtime, over 1 million times observations excluding V8 initialization, of the two functions in the base (white), CT bytecode-only (light grey) and CT-bytecode-handler (dark grey) configurations. **Scooti** introduces an overhead of 8% and 64% for *select* and *indcall*, respectively, which is mainly due to the CT bytecode. The original bytecode of *select* and *indcall* contains 9 and 16 instructions, respectively, which increases to 16 and 34, respectively, in the CT bytecode (Fig 5).

Future work. To address C2, we will use information flow analysis within the bytecode and the V8 engine to avoid costs of CT transformations on program paths that are secret-independent. To address C3, we plan to adopt ORAM-style techniques [13] into the V8 memory management, which would mitigate leaks via memory access patterns at a lower cost than naïvely iterating through complex datastructures.

We plan to provide a formal proof or a runtime type system to validate that our transformation and interpretation compose to achieve overall constant-time execution.

We will further empirically evaluate the security and performance of our solution when adopted to real-world applications (e.g., libjpeg, spell checker). For security, we will show the accuracy of attacks against the applications in absence and presence of **Scooti**. For performance, we will evaluate the overheads of various components of **Scooti**, and extend our performance microbenchmarks to evaluate the end-to-end runtime overheads for the real-world applications.

References

- [1] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. 2017. Jasmin: High-Assurance and High-Speed Cryptography. *ACM CCS* (2017).
- [2] Gilles Barthe, Marco Gaboardi, Emilio Jesús Gallego Arias, Justin Hsu, Aaron Roth, and Pierre-Yves Strub. 2016. Computer-Aided Verification for Mechanism Design. *Web and Internet Economics* (2016).
- [3] Pietro Borrello, Daniele Cono D'Elia, Leonardo Querzoni, and Cristiano Giuffrida. 2021. Constantine: Automatic Side-Channel Resistance Using Efficient Control and Data Flow Linearization. *ACM CCS* (2021).
- [4] Benjamin A. Braun, Suman Jana, and Dan Boneh. 2015. Robust and Efficient Elimination of Cache and Timing Side Channels. *CoRR* abs/1506.00189 (2015).
- [5] Tegan Brennan, Nicolás Rosner, and Tevfik Bultan. 2020. JIT Leaks: Inducing Timing Side Channels through Just-In-Time Compilation. In *2020 IEEE Symposium on Security and Privacy (SP)*.
- [6] Sunjay Cauligi, Gary Soeller, Brian Johannesmeyer, Fraser Brown, Riad S. Wahby, John Renner, Benjamin Grégoire, Gilles Barthe, Ranjit Jhala, and Deian Stefan. 2019. FaCT: A DSL for Timing-Sensitive Computation. *ACM PLDI* (2019).
- [7] Lesly-Ann Daniel, Sébastien Bardin, and Tamara Rezk. 2020. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level. *IEEE S&P* (2020).
- [8] Jan Jancar, Marcel Fourné, Daniel De Almeida Braga, Mohamed Sabt, Peter Schwabe, Gilles Barthe, Pierre-Alain Fouque, and Yasemin Acar. 2022. "They're not that hard to mitigate": What Cryptographic Library Developers Think About Timing Attacks. *IEEE S&P* (2022).
- [9] Paul C. Kocher. 1996. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. *Advances in Cryptology — CRYPTO*.
- [10] Fangfei Liu, Qian Ge, Yuval Yarom, Frank Mckeen, Carlos Rozas, Gernot Heiser, and Ruby B. Lee. 2016. CATalyst: Defeating last-level cache side channel attacks in cloud computing. *IEEE HPCA* (2016).
- [11] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache Attacks and Countermeasures: The Case of AES. *Topics in Cryptology — CT-RSA* (2006).
- [12] Qi Qin, JulianAndres JiYang, Fu Song, Taolue Chen, and Xinyu Xing. 2022. DeJITLeak: Eliminating JIT-Induced Timing Side-Channel Leaks. *ACM ESEC/FSE* (2022).
- [13] Ashay Rane, Calvin Lin, and Mohit Tiwari. 2015. Raccoon: Closing Digital Side-Channels through Obfuscated Execution. *USENIX Security* (2015).
- [14] Jicheng Shi, Xiang Song, Haibo Chen, and Binyu Zang. 2011. Limiting cache-based side-channel in multi-tenant cloud using dynamic page coloring. *IEEE/IFIP Intl. Conf. on Dependable Systems and Networks Workshops (DSN-W)* (2011).
- [15] Anatoly Shusterman, Ayush Agarwal, Sioli O'Connell, Daniel Genkin, Yossi Oren, and Yuval Yarom. 2021. Prime+Probe 1, JavaScript 0: Overcoming Browser-based Side-Channel Defenses. *USENIX Security* (2021).
- [16] Venkatanathan Varadarajan, Thomas Ristenpart, and Michael M Swift. [n. d.]. Scheduler-based Defenses against Cross-VM Side-channels. *USENIX Security*, 2014 ([n. d.]).
- [17] Conrad Watt, John Renner, Natalie Popescu, Sunjay Cauligi, and Deian Stefan. 2019. CT-Wasm: Type-Driven Secure Cryptography for the Web Ecosystem. *ACM POPL* (2019).
- [18] Yuval Yarom and Katrina Falkner. 2014. FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. *USENIX Security* (2014).
- [19] Yingqian Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. 2012. Cross-VM Side Channels and Their Use to Extract Private Keys. *ACM CCS* (2012).

```
function select(secret, val) {
  if (secret != 0) return val;
}

function indcall(secret_index) {
  return funclist[secret_index]();
}
```

Figure 4. *select* and *indcall*

```
1 function select(secret, val) {
2   if (secret != 0)
3     return val;
4 }
----- original -----
1 LdaZero      /* Line 2 */
2 TestEqual a0, [0]
3 JumpIfTrue [5]
4 Ldar a1      /* Line 3 */
5 Return
6 LdaUndefined /* Line 4 */
7 Return

----- CT -----
1 LdaZero
2 TestEqual a0, [0]
3 Star r0
4 Mov a1, r1
5 LdaUndefined
6 Star r2
7 Ldar r0
8 CTSelect r2, r1
9 Return
```

Figure 5. Original and CT bytecode of *select*